

Size Balanced Tree

Chen Qifeng (Farmer John)

2007

Nguồn gốc: Size Balanced Tree

IOI China candidate team papers / 2007 / day 1 / Chen Qifeng.ppt

Tóm tắt

Size Balanced Tree (SBT) là cây tìm kiếm nhị phân được giữ cân bằng bằng kích thước cây con. Mỗi nút lưu hai con, khóa và kích thước cây con; các phép quay giữ nguyên thứ tự inorder nhưng khôi phục các ràng buộc cân bằng. Bản trình chiếu tập trung vào bất biến của SBT, thủ tục MAINTAIN, các thao tác chèn/xóa/tìm hạng và phân tích vì sao chiều cao là $\mathcal{O}(\log n)$.

Từ khóa. Size Balanced Tree; cây tìm kiếm nhị phân; quay cây; hạng; chọn phần tử.

1 Cây tìm kiếm nhị phân và phép quay

Trong một cây tìm kiếm nhị phân, với mỗi nút x , mọi khóa ở cây con trái không lớn hơn khóa của x , còn mọi khóa ở cây con phải không nhỏ hơn khóa của x . SBT dùng các mảng hoặc trường:

$$left[t], \quad right[t], \quad key[t], \quad s[t],$$

trong đó $s[t]$ là số nút trong cây con gốc t . Con trỏ rỗng được ký hiệu là 0, với $s[0] = 0$.

Phép quay trái và quay phải là thao tác cục bộ thay đổi một số hằng định con trỏ. Chúng giữ nguyên thứ tự inorder, nên tính chất tìm kiếm nhị phân không đổi. Sau mỗi phép quay phải cập nhật lại kích thước:

$$s[t] = s[left[t]] + s[right[t]] + 1.$$

Mã giả trong slide chỉ gồm các phép gán con trỏ chuẩn của quay trái/quay phải; theo ngoại lệ dành cho mẫu mã dài, bản ghi chú này không chép lại toàn bộ listing.

2 Bất biến của SBT

Gọi T là một nút, L là con trái, R là con phải. Gọi A, B là hai cây con của L , và C, D là hai cây con của R . SBT duy trì hai nhóm ràng buộc:

$$s[A], s[B] \leq s[R], \quad s[C], s[D] \leq s[L].$$

Viết bằng chỉ số mảng:

$$\begin{aligned}s[\text{left}[\text{left}[t]]] &\leq s[\text{right}[t]], \\ s[\text{right}[\text{left}[t]]] &\leq s[\text{right}[t]], \\ s[\text{right}[\text{right}[t]]] &\leq s[\text{left}[t]], \\ s[\text{left}[\text{right}[t]]] &\leq s[\text{left}[t]].\end{aligned}$$

Trực giác là: cây con của một phía không được lớn hơn toàn bộ phía đối diện. Điều kiện này yếu hơn cân bằng tuyệt đối, nhưng đủ để bảo đảm chiều cao logarit và có thể sửa bằng một số ít phép quay.

3 MAINTAIN

Sau khi chèn hoặc xóa, bất biến có thể bị phá. Thủ tục trung tâm của SBT là MAINTAIN. Giả thiết trước khi gọi là hai cây con của nút hiện tại đã là SBT; thủ tục chỉ cần sửa mất cân bằng quanh nút đó.

Có bốn trường hợp:

1. $s[\text{left}[\text{left}[t]]] > s[\text{right}[t]]$. Cây con trái-trái quá lớn; quay phải tại t , rồi duy trì các phần bị ảnh hưởng.
2. $s[\text{right}[\text{left}[t]]] > s[\text{right}[t]]$. Cây con trái-phải quá lớn; quay trái tại con trái, rồi quay phải tại t .
3. $s[\text{right}[\text{right}[t]]] > s[\text{left}[t]]$. Đối xứng với trường hợp 1; quay trái tại t .
4. $s[\text{left}[\text{right}[t]]] > s[\text{left}[t]]$. Đối xứng với trường hợp 2; quay phải tại con phải, rồi quay trái tại t .

Slide đưa hai phiên bản mã giả: bản chuẩn kiểm tra cả bốn trường hợp, và bản nhanh hơn có tham số *flag*. Khi *flag* cho biết phía vừa thay đổi, ta chỉ cần kiểm tra hai trường hợp có thể bị phá, tránh các kiểm tra thừa. Sau khi quay, thủ tục gọi lại trên các con cần thiết và cuối cùng gọi lại trên chính nút hiện tại với hai giá trị *flag*.

4 Chèn

Chèn trong SBT giống chèn BST thông thường: đi từ gốc theo so sánh khóa, tăng $s[t]$ trên đường đi, rồi đặt nút mới tại vị trí rỗng. Khi quay lui đệ quy, gọi MAINTAIN tại các nút trên đường. Tham số *flag* thường được đặt theo việc khóa mới đi sang phải hay sang trái của nút hiện tại.

Vì mỗi tầng chỉ xử lý một số hằng định phép quay và chiều cao là logarit, thời gian chèn là $\mathcal{O}(\log n)$ theo khẩu hao.

5 Xóa

Slide trình bày hai cách xóa. Cách chuẩn tìm nút cần xóa; nếu nút có hai con, thay khóa bằng phần tử kế cận rồi xóa phần tử kế cận ở cây con. Sau đó gọi MAINTAIN để khôi phục cân bằng. Cách này giữ cấu trúc SBT đúng theo bất biến sau mỗi lần xóa.

Phiên bản nhanh hơn đơn giản hóa mạnh: nếu giá trị cần xóa không tồn tại, nó xóa nút cuối cùng gặp trên đường tìm kiếm; nếu nút có một con thì thay nút bằng con đó, nếu có hai con thì thay bằng một giá trị lấy từ cây con. Phiên bản này có thể tạm thời không duy trì đủ bất biến sau từng bước, nhưng khi kết hợp với chèn chuẩn, chiều cao vẫn được chặn bởi $\mathcal{O}(\log n^*)$, trong đó n^* là tổng số lần chèn đã xảy ra.

6 Các thao tác thứ tự

Nhờ trường kích thước $s[t]$, SBT hỗ trợ trực tiếp các thao tác trên thứ tự:

RANK(t, v) trả về 1 cộng với số khóa nhỏ hơn v . Khi đi sang phải từ một nút, cộng thêm kích thước cây con trái và chính nút đó.

SELECT(t, k) tìm phần tử đứng thứ k . Nếu $k = s[\text{left}[t]] + 1$, đáp án là t ; nếu nhỏ hơn thì đi trái, nếu lớn hơn thì trừ phần đã bỏ qua và đi phải.

PRED(t, v) tìm khóa lớn nhất nhỏ hơn v bằng cách đi theo BST và lưu ứng viên cuối cùng nhỏ hơn v .

SUCC(t, v) đối xứng, tìm khóa nhỏ nhất lớn hơn v .

Tất cả các thao tác này chỉ đi theo một đường từ gốc nên tốn $\mathcal{O}(\log n)$.

7 Phân tích chiều cao

Gọi $f[h]$ là số nút ít nhất của một SBT có chiều cao h . Từ bất biến kích thước, nếu một cây con có chiều cao $h - 1$, phía đối diện phải đủ lớn để che được cây con chiều cao $h - 2$. Vì vậy

$$f[0] = 1, \quad f[1] = 2, \quad f[h] = f[h - 1] + f[h - 2] + 1 \quad (h > 1).$$

Đây là truy hồi Fibonacci. Do $f[h]$ tăng theo hàm mũ, chiều cao lớn nhất của SBT có n nút là $\mathcal{O}(\log n)$. Bản gốc ghi cận xấp xỉ

$$h \leq 1.44 \log_2 n + O(1).$$

8 Phân tích MAINTAIN

Một cách đánh giá chất lượng cây là tổng độ sâu SD của mọi nút. Trong các trường hợp mà MAINTAIN thực sự quay, SD giảm: ví dụ ở trường hợp trái-trái, lượng thay đổi liên quan đến

$$s[\text{right}[t]] - s[\text{left}[\text{left}[t]]],$$

là âm đúng vì điều kiện mất cân bằng đang xảy ra. Sau một lần chèn, SD chỉ tăng $\mathcal{O}(\log n)$; các phép quay trong MAINTAIN tiêu bớt phần tăng này. Từ đó suy ra số lần quay trung bình trên mỗi thao tác là hằng số theo khẩu hao, còn đường đi tìm kiếm vẫn dài $\mathcal{O}(\log n)$.

9 Ưu điểm thực tế

SBT có ưu thế trong thi lập trình vì cài đặt ngắn và các thao tác thứ tự không cần cấu trúc phụ. So với treap, SBT không cần số ngẫu nhiên; so với AVL hoặc red-black tree, bất biến kích thước đồng thời phục vụ cân bằng và truy vấn hạng. Nhược điểm là MAINTAIN dễ viết sai ở các trường hợp quay kép và cập nhật kích thước. Khi cài đặt, nên kiểm tra kỹ các công thức:

$$s[0] = 0, \quad s[t] = s[\text{left}[t]] + s[\text{right}[t]] + 1$$

sau mọi phép quay, chèn và xóa.

10 Kết luận

Size Balanced Tree là một BST cân bằng dựa trên kích thước, hỗ trợ INSERT, DELETE, FIND, RANK, SELECT, PRED và SUCC trong $\mathcal{O}(\log n)$. Điểm cốt lõi là bất biến so sánh kích thước cây cháu với cây anh em, cùng thủ tục MAINTAIN sửa bốn dạng vi phạm bằng quay đơn hoặc quay kép.